

# Appunti — Modulo S1: Principi di Offensive Security e Enumerazione

**Corso:** Lab Sicurezza Informatica T **Materiale:** Principi\_delloffensive\_security\_20\_febbraio.pdf  
· Introduzione\_alla\_materia\_20\_febbraio.pdf **Lab:** guida\_lab\_moduloS1\_enumerazione\_nmap.md  
**Stato:** lezione · grezzi · appunti · lab

---

## 1. Cos'è l'offensive security

**Offensive security:** simulazione di attacchi reali, con permesso e obiettivi definiti, per trovare le falle *prima* di un attaccante ostile. Il difensore che non ha mai “pensato da attaccante” tende a lasciare aperture ovvie proprio dove non guarda.

**VA (Vulnerability Assessment):** enumera le vulnerabilità note del sistema. Risponde a “cosa c'è di rotto?”.

**PT (Penetration Testing):** assessment autorizzato che *sfrutta* le falle trovate dalla VA, verifica le catene di attacco, e produce un report che dimostra l'impatto reale. Risponde a “cosa riesco a fare davvero con le falle?”. “Autorizzato” è la parola chiave: le stesse tecniche senza autorizzazione sono reati.

**Red Team:** operazione prolungata (settimane o mesi) che simula un attore ostile sofisticato, incluse tecniche di social engineering e movimenti laterali, senza che il blue team sappia quando e come arriverà l'attacco.

**Metodologie standardizzate:** OSSTMM, OWASP (focalizzato su web app), NIST 800-115 (guida tecnica governo USA), ISSAF, PCI DSS (sistemi di pagamento). Garantiscono che il PT sia riproducibile, misurabile, e difendibile legalmente.

Questa sezione non era presente negli appunti grezzi — MITRE ATT&CK. **MITRE ATT&CK** è il catalogo delle tecniche usate in ciascuna fase da gruppi APT reali. È la “biblioteca” che i team di difesa consultano per sapere cosa aspettarsi da un attore specifico. Compare spesso nel quiz teorico come riferimento alla nomenclatura delle tecniche.

---

## 2. Kill Chain

Ottima intuizione: avevi il nome corretto. La struttura è questa:

La **Kill Chain** è la sequenza di fasi che compone un attacco completo:

| Fase                         | Cosa succede                                                |
|------------------------------|-------------------------------------------------------------|
| <b>Reconnaissance</b>        | Raccolta info sull'obiettivo (passiva e attiva)             |
| <b>Weaponization</b>         | Costruzione del payload/exploit                             |
| <b>Delivery</b>              | Consegna del payload al target (phishing, web, USB)         |
| <b>Exploitation</b>          | Esecuzione dell'exploit sulla vulnerabilità                 |
| <b>Installation</b>          | Installazione di backdoor/RAT per persistenza               |
| <b>Command &amp; Control</b> | Connessione al C2 server per controllare l'host compromesso |

| Fase                         | Cosa succede                                                        |
|------------------------------|---------------------------------------------------------------------|
| <b>Actions on Objectives</b> | Raggiungimento dell'obiettivo (data exfiltration, ransomware, etc.) |

**In S1 si affrontano solo le prime due fasi** — Reconnaissance ed Enumeration. Le fasi successive entrano in S3-S11.

### 3. Rischio, difesa e superfici d'attacco

Ottima definizione: “rischio come prodotto tra probabilità e impatto” — è proprio la formula della lezione.

**RISCHIO = PROBABILITÀ × IMPATTO.** Orienta quanto sforzo vale la pena investire per correggere una falla.

“Window of Exposure: tempo che passa tra la scoperta della falla e la patch” — corretto. Si mira a tenerla breve.

**Zero-day:** vulnerabilità sfruttata prima che esista una patch — window of exposure teoricamente infinita per la vittima.

**Paesaggio minacce reali** (ENISA 2023): ransomware 31%, DDoS 21%, violazioni dati 20%. Tre casi concreti che illustrano la complessità moderna:

Questa sezione non era presente negli appunti grezzi — casi concreti. - **Stuxnet:** malware che ha sabotato fisicamente le centrifughe di arricchimento uranio iraniane modificando i PLC Siemens. Primo caso documentato di attacco cyber con effetti fisici nel mondo reale. - **SolarWinds (2020):** attacco supply chain. Gli attaccanti hanno compromesso il processo di build di Orion inserendo una DLL malevola distribuita come aggiornamento legittimo firmato dal vendor a migliaia di organizzazioni. - **xz/liblzma (CVE-2024-3094):** due anni di social engineering su un maintainer open source per inserire una backdoor in una libreria presente in quasi tutte le distro Linux. La supply chain open source è vulnerabile quanto quella commerciale.

#### NIST Cybersecurity Framework (CSF):

La struttura che chiedevi — “argomenta e inserisci il processo”:

| Fase            | Domanda a cui risponde                       |
|-----------------|----------------------------------------------|
| <b>Identify</b> | Cosa ho? Cosa vale la pena proteggere?       |
| <b>Protect</b>  | Come riduco la probabilità di un attacco?    |
| <b>Detect</b>   | Come mi accorgo che qualcosa sta succedendo? |
| <b>Respond</b>  | Cosa faccio quando rilevo un incidente?      |
| <b>Recover</b>  | Come ripristino l'operatività dopo?          |

Non è lineare ma **ciclico**: un incidente alimenta la fase Identify del ciclo successivo.

**Politiche & Meccanismi:** le politiche dichiarano *cosa* è consentito (“nessun accesso remoto senza MFA”); i meccanismi sono gli strumenti tecnici che le fanno rispettare (firewall, IDS, autenticazione). Una politica senza meccanismo è carta; un meccanismo senza politica è rumore.

## Superfici d'attacco — 3 vettori:

“Forme di attacco: fisico, cyber, umano” — corretto e completo.

- **Fisico:** accesso fisico ai locali e alle macchine
  - **Cyber:** rete, applicazioni, sistemi (focus di S1)
  - **Umano:** phishing, social engineering (si intreccia lungo tutto il programma)
- 

## 4. Reconnaissance

La reconnaissance è la fase in cui si raccolgono informazioni sull'obiettivo **senza interagire direttamente con i suoi sistemi**. Si divide in: - **Passiva:** fonti pubbliche, nessun contatto col target. Lascia meno tracce. Si inizia sempre da qui. - **Attiva:** interazione diretta (ping, port scan). Più informazioni, più rumore.

### 4.1 Google Dorking

La tabella degli operatori che chiedevi:

Google indicizza tutto ciò che viene esposto, volontariamente o per errore. Un attaccante lo usa come chiunque altro.

| Operatore              | Cosa fa                           | Esempio                        |
|------------------------|-----------------------------------|--------------------------------|
| <code>site:</code>     | Limita la ricerca a un dominio    | <code>site:unibo.it</code>     |
| <code>filetype:</code> | Tipo di file specifico            | <code>filetype:PDF</code>      |
| <code>intext:</code>   | Parola nel contenuto della pagina | <code>intext:password</code>   |
| <code>intitle:</code>  | Parola nel titolo della pagina    | <code>intitle:index.of</code>  |
| <code>inurl:</code>    | Parola nell'URL                   | <code>inurl:admin</code>       |
| <code>cache:</code>    | Versione in cache di Google       | <code>cache:example.com</code> |
| <code>- (minus)</code> | Esclude un termine                | <code>-id_rsa.pub</code>       |

Esempi concreti dalle slide:

```
site:ulisse.unibo.it filetype:PDF intext:password  
→ trova PDF su quel dominio che contengono la parola "password"
```

```
site:ulisse.unibo.it intitle:index.of id_rsa -id_rsa.pub  
→ cerca directory listing con chiavi SSH private esposte
```

Il database **Google Hacking** ([exploit-db.com/google-hacking-database](http://exploit-db.com/google-hacking-database)) cataloga dork già noti per trovare configurazioni errate comuni.

“robots.txt: convenzione tra web developer e crawler legittimi... un hacker se ne sbatte”  
— ottima sintesi del punto critico.

**robots.txt** dice ai crawler cosa non indicizzare. **Disallow: /cartella-sensibile** blocca i crawler legittimi — ma è una **convenzione, non un meccanismo di sicurezza**. Un crawler ostile lo ignora, e la stessa lista delle cartelle escluse può rivelare all'attaccante cosa c'è di interessante. La vera difesa è non esporre mai dati sensibili su server pubblici.

## 4.2 OSINT su IP e domini

Gerarchia di assegnazione IP: **IANA** → **RIR** (es. RIPE per l'Europa) → **LIR** (università, ISP). Conoscere anche solo un IP di un sistema dell'obiettivo permette di risalire via RIPE a tutti i blocchi allocati a quell'organizzazione.

## 4.3 DNS Enumeration

Il “retro engineering” che intuiti è esatto come concetto — è proprio quello che si fa: da un IP si risale all'organizzazione. La sezione DNS che chiedevi:

Il DNS è l'elenco telefonico di internet, ma anche una miniera di informazioni organizzative. I **record DNS** rivelano struttura e infrastruttura del target:

| Tipo  | Cosa rivela                                            |
|-------|--------------------------------------------------------|
| A     | hostname → IPv4                                        |
| AAAA  | hostname → IPv6                                        |
| CNAME | alias (rivela nomi interni, CDN usati)                 |
| MX    | mail server dell'organizzazione                        |
| NS    | nameserver autoritativi (rivela provider DNS, cloud)   |
| SOA   | zona + email admin del dominio                         |
| TXT   | SPF → mappa infrastruttura cloud                       |
| PTR   | reverse lookup: da IP a hostname (rivela nomi interni) |
| SRV   | service locator (es. SIP, XMPP)                        |

Tool: `nslookup google.com` (record A base); `nslookup -type=any google.com` (tutti i tipi); `dnsrecon -d DOMAIN`; `dnsmap DOMAIN` (bruteforce sottodomini).

**Zone transfer (AXFR):** se un nameserver è mal configurato, risponde a richieste AXFR restituendo l'intera zona DNS in una query — tutta la struttura interna dell'organizzazione in un colpo solo.

## 4.4 Subdomain Enumeration e CT Abuse

La versione semplificata che chiedevi:

I sottodomini sono superfici d'attacco trascurate — dietro `old-staging.example.com` può nascondersi un'applicazione dimenticata, non aggiornata.

**Distinzione critica:** `http://myaltropc.ulisse.com` non è un subdomain DNS — è un'applicazione ospitata sulle porte 80/443 di un host. Un subdomain DNS come `corsosec.ulisse.com` potrebbe invece esporre un servizio su porta 8080 senza nulla su 80. Strategia corretta: (1) mappa tutti i subdomain DNS, (2) per ciascuno analizza separatamente quali servizi espone e su quali porte.

**Approccio passivo** — interroga dataset pubblici senza toccare il target: - SecurityTrails, Shodan, Censys, VirusTotal, Binaryedge - Tool: `amass` (OWASP), `subfinder`, `assetfinder`, `Findomain`

**Certificate Transparency (CT) abuse:** ogni certificato TLS emesso viene registrato in log pubblici e immutabili (RFC 6962). Interrogando `crt.sh/?q=%25.example.com` si ottengono tutti

i sottodomini che hanno mai avuto un certificato — inclusi quelli rimossi dal DNS ma ancora raggiungibili. Non esiste difesa: la trasparenza è by design.

**Approccio attivo:** DNS bruteforcing, permutation/alternations, VHOST probing. Non esiste difesa reale dall'enumeration attiva — i subdomain sono pubblici per definizione.

**Shodan.io:** motore di ricerca per dispositivi connessi. Mentre Google indicizza HTML, Shodan indicizza i banner dei servizi esposti su internet: porte aperte, versione del software, geolocalizzazione, organizzazione.

---

## 5. nmap — Enumerazione Attiva

La spiegazione con esempi e output di sistema che chiedevi:

nmap è lo strumento centrale della fase di enumerazione attiva. Si usa in tre passaggi progressivi:

### 5.1 Host Discovery (-sn)

```
sudo nmap -sn 192.168.X.0/24
```

```
Nmap scan report for 192.168.56.32
Host is up (0.007s latency).
Nmap scan report for 192.168.56.33
Host is up (0.001s latency).
```

**Errore frequente:** nmap -sn senza sudo — senza privilegi non usa ARP su rete locale, più lento e meno affidabile.

### 5.2 Port Scan completo (-p-)

```
nmap -sT 192.168.56.32 -p-
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
3306/tcp  open  mysql
5432/tcp  open  postgresql
```

**Errore frequente:** senza -p- nmap scansiona solo ~1000 porte popolari. Un servizio SSH su 1337 non appare. Usa sempre -p- per la mappa completa.

**Differenza -sT vs -sS:** -sT completa il three-way handshake (non richiede root, più rumoroso); -sS invia solo SYN e non completa la connessione (richiede root, più veloce e stealth, default con sudo).

### 5.3 Version Detection (-sV)

```
nmap -sV 192.168.56.32 -p 22,80,3306,5432
```

```
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4
80/tcp    open  http     Apache httpd 2.4.57
```

```
3306/tcp open  mysql      MariaDB 10.11.4
5432/tcp open  postgresql PostgreSQL 15.3
```

**Errore frequente:** `-sT` e `-sV` non sono equivalenti. `-sT` = TCP connect, dice solo “porta aperta/chiusa”. `-sV` legge il banner del servizio e identifica nome + versione. Per l’enumeration vera serve `-sV`.

**Errore frequente:** `-p 22 80 3306` — spazi tra i numeri fanno trattare 80 e 3306 come host aggiuntivi. Usa sempre la virgola: `-p 22,80,3306`.

**unicornsanscan:** alternativa a nmap con fingerprinting più affidabile e più veloce (separa fase di invio e ricezione). Preferibile quando si vuole evitare il **reverse fingerprinting** (il target non riesce a identificare l’OS dell’attaccante dal modo in cui costruisce i pacchetti).

---

## 6. Evasione: scan senza essere visti

Questa sezione non era presente negli appunti grezzi.

La fase di enumeration genera traffico visibile — IDS e IPS possono rilevarlo.

- **Reconnaissance anonima:** ToR, account usa-e-getta, VM periodicamente sostituite
- **Timing nmap:** `-T0` (ultra-lento, stealth) → `-T5` (aggressivo). Default: `-T3`
- **Stealth scan:** randomizza ordine IP e porte, usa decoy scan, source port spoofing
- **Reverse fingerprinting:** il target identifica l’OS dell’attaccante dal modo in cui costruisce i pacchetti TCP. unicornsanscan lo evita usando stack separato

---

## 7. Tentativi di accesso ai servizi e Vulnerability Scanners

Questa sezione non era presente negli appunti grezzi.

Dopo enumeration, si analizzano i **protocolli applicativi** esposti: SMB, SMTP, SNMP. Il **fuzzing applicativo** (tool: `bed`, `doona`) invia payload randomizzati per trovare crash o info leak.

**Vulnerability scanners** — automatizzano il passaggio da “porta aperta” a “vulnerabilità sfruttabile”: - **Nessus** (Tenable, commerciale) - **OpenVAS / Greenbone Community Edition (GCE)** — fork open-source di Nessus dal 2005. Architettura: OpenVAS Scanner + Notus Scanner → `osspd-openvas` → `gvmd` → interfaccia web **Greenbone Security Assistant (GSA)**. Il feed giornaliero contiene **NVT** (Network Vulnerability Tests): per ogni vulnerabilità, descrizione, piattaforme colpite, processo di verifica.

---

## 8. Postura interna vs esterna

Questa sezione non era presente negli appunti grezzi.

Reconnaissance ed enumeration si fanno tipicamente **dall’esterno** — postura fedele all’attaccante reale. Dall’esterno però ci sono NIDS e firewall di perimetro.

**Auto-attacco dall'interno:** si simula un attaccante già infiltrato. NIDS e FW perimetrali vengono scavalcati, i sistemi interni sono raggiungibili direttamente, i test per HIDS (S11) sono più efficaci. In S1 si parte dall'esterno; S7–S11 esplorano la postura interna.

---

## 9. Connessioni

- **SysAdmin 3D:** `ss -tlnp` dall'interno mostra le stesse porte che nmap vede dall'esterno — stessa realtà, prospettiva ribaltata
  - **S2 (Autenticazione):** nmap `-sV` rivela i servizi; S2 mostra come attaccarli a livello di credenziali
  - **S3 (Web Security):** la subdomain enumeration porta a web app dimenticate — quelle stesse app vengono attaccate in S3 con OWASP Top 10
  - **S5 (Firewall):** la scansione stealth di nmap è il punto di vista opposto rispetto alle regole iptables di S5
  - **S10 (Suricata):** ogni scan nmap genera traffico rilevabile. In S1 lo si genera; in S10 si scrivono regole per rilevarlo
- 

## 10. Domande di autoverifica — Risposte

**Tutte e 6 le risposte sono corrette!**

1. Un PT differisce da una VA perché: **C** — verifica effettivamente la sfruttabilità delle vulnerabilità trovate.
  2. `site:example.com filetype:PDF intext:password` trova: **B** — PDF su example.com che contengono la parola “password” nel testo.
  3. Il record DNS di tipo MX rivela: **B** — i mail server dell'organizzazione. > Avevi il (?) — conferma: B è corretto. MX = Mail eXchanger.
  4. `-sT` e `-sV` forniscono lo stesso livello di informazione: **Falso**. > `-sT` = porta aperta/chiusa; `-sV` = porta + servizio + versione via banner.
  5. La CT abuse permette di: **B** — enumerare i sottodomini cercando nei log CT pubblici.
  6. `robots.txt` con `Disallow: /` impedisce agli attaccanti di accedere alle cartelle escluse: **Falso**. > Il tuo “Fc” era abbreviazione di “Falso” — risposta esatta.
- 

## Collegati

- [\[\[guida\\_lab\\_moduloS1\\_enumerazione\\_nmap\]\]](#)
- [\[\[lezione\\_moduloS1\\_offensive\\_security\\_enumerazione\]\]](#)

**Hub:** [\[\[master\\_map\\_studio\]\]](#) · [\[\[concept\\_maps\]\]](#) · [\[\[metodo\\_studio\\_esami\\_pratici\]\]](#)